

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Governance	Documentation quality	High	
Timeline	2025-12-10 through 2025-12-15	Test quality	High	
Language	Solidity	Total Findings	2	Fixed: 2
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	Audit Scope - Risk Stewards ⓘ	Medium severity findings ⓘ	0	
Source Code	https://github.com/VenusProtocol/governance-contracts ⓘ #2d81a84 ⓘ	Low severity findings ⓘ	2	Fixed: 2
Auditors	- Ibrahim Abouzied Auditing Engineer - Ed Zulkoski Senior Auditing Engineer - Cameron Biniamow Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	0	

Summary of Findings

This audit encompasses the Venus Risk Stewarding System: A modular framework for handling risk-parameter updates without requiring Governance proposals. Approved third parties send suggested parameter updates to the RiskOracle . Any user can then notify the RiskStewardReceiver of the new parameter suggestion and register the update. The RiskStewardReceiver will then call the relevant IRiskSteward to update the underlying Venus contract. The IRiskSteward determines whether to immediately execute updates deemed safe, or have a whitelisted Executor execute the registered update after a timelock. The RiskStewardReceiver can also send cross-chain messages using LayerZero to a DestinationStewardReceiver , where parameter updates can be processed for remote deployments in a similar fashion.

The audit uncovered two low-severity issues. Both the source and destination chains enforce separate debounce periods for the same transaction, which could lead to operational difficulty (VEN-1). Some of the Risk Stewards do not check the return value of their parameter updates, in which case the update may silently fail (VEN-2). We also make a few recommendations to improve code quality.

Fix Review Update: The Venus team has fixed all of the findings and fixed or acknowledged all of the auditor suggestions.

ID	DESCRIPTION	SEVERITY	STATUS
VEN-1	Multiple Debounce for Cross-Chain Calls	• Low ⓘ	Fixed
VEN-2	Parameter Updates May Silently Fail Due to Unchecked Return Values	• Low ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

- `contracts/interfaces/ICorePoolComptroller.sol`
- `contracts/interfaces/IILVToken.sol`
- `contracts/interfaces/IIsolatedPoolsComptroller.sol`
- `contracts/interfaces/IVToken.sol`
- `contracts/RiskSteward/Interfaces/IRiskOracle.sol`
- `contracts/RiskSteward/Interfaces/IRiskSteward.sol`
- `contracts/RiskSteward/Interfaces/IRiskStewardReceiver.sol`
- `contracts/RiskSteward/CollateralFactorsRiskSteward.sol`
- `contracts/RiskSteward/DestinationStewardReceiver.sol`
- `contracts/RiskSteward/IRMRiskSteward.sol`
- `contracts/RiskSteward/MarketCapsRiskSteward.sol`
- `contracts/RiskSteward/RiskOracle.sol`
- `contracts/RiskSteward/RiskStewardReceiver.sol`

Repo: `https://github.com/VenusProtocol/governance-contracts`

Included Paths: `contracts/RiskSteward`

Operational Considerations

- **Access Controls:** Contract functions that the Access Control Manager guards require the default admin role to assign the necessary roles manually. Failure to grant the roles will prevent execution of the respective functions.
- **Upgradeable Contracts:** All in-scope contracts are upgradeable. Any contract implementation upgrades should be audited to ensure the logic functions as intended and there are no storage collisions.
- **LayerZero Configuration:** The `RiskStewardReceiver` contract sends cross-chain messages via LayerZero to `DestinationStewardReceiver` on remote chains. For successful execution, the `RiskStewardReceiver` owner must properly set the remote `DestinationStewardReceiver` contracts as peers with their respective endpoint ID.
- **Bridge Fees:** The `RiskStewardReceiver` must hold a sufficient balance of native tokens to cover the bridge fees for remote updates. An insufficient balance will cause the `processUpdate()` function to fail until the protocol or the caller directly funds the contract.

- **Replayed Remote Updates:** Remote updates from the `RiskStewardReceiver` to the `DestinationStewardReceiver` are replayable by whitelisted executors even if the update was successfully bridged and executed on the remote chain. Replaying successful remote updates will incur excess LayerZero fees, which the `RiskStewardReceiver` contract may pay if the caller does not forward any native tokens.
- **Steward Key-space:** The key-space for Update Types is shared across all stewards. If two stewards support update types with identical names, they will be forced to share the same `RiskParamConfig`.
- **Timestamp Dependence:** The `DestinationStewardReceiver` uses the `update.timestamp` field to validate whether an `update` has expired. This assumes that the source and destination chains have the same `block.timestamp` at T=0, or that they will be with an acceptable degree of precision.
- **Market and Update Type Key-Space:** Many 2d mappings store updates under the indexes `[updateTypeKey][market]`. This does not distinguish between which chain the market is deployed on. If a market with the same address and update type is deployed on a remote chain, it would cause a collision in these mappings. The team has confirmed that no such deployments are planned.

Key Actors And Their Capabilities

Access Control Manager

A centralized contract used in the Venus protocol, which manages access controls per account for specified contract functions via function signatures. The default admin role grants and revokes the roles as needed. The Access Control Manager is used by all risk steward contracts.

Risk Oracle

A contract that manages and publishes risk parameter updates. After updates are published, anyone can register the update in the Risk Steward Receiver.

- **Risk Oracle Contract Owner:** Can update the Access Control Manager contract address; transfer the owner role to another account; and renounce their role, leaving the contract without an owner.
- **Risk Oracle Access Control Manager Roles:**
 - `addAuthorizedSender()` : Authorizes accounts that can perform updates.
 - `removeAuthorizedSender()` : Revokes authorization from accounts, preventing them from performing updates.
 - `addUpdateType()` : Creates a new update type and sets its status as active.
 - `setUpdateTypeActive()` : Activates or deactivates the status of an existing update type.
- **Risk Oracle Authorized Senders:** Can publish risk parameter updates for a given market and update type.

Risk Steward Receiver

The entry point contract where updates are registered, executed, or rejected. Updates are fetched from the Risk Oracle once published and can be registered by anyone; however, this is typically handled by Venus Keeper bots.

- **Risk Steward Receiver Contract Owner:** Can update the Access Control Manager contract address; claim the full native token balance in the `RiskStewardReceiver` contract; set peers for corresponding LayerZero endpoint IDs; update the delegate address, who can set LayerZero configurations directly in the endpoint contract; and transfer the owner role to another account.
- **Risk Steward Receiver Access Control Manager Roles:**
 - `setPaused()` : Sets the pause status to enable or disable update processing.
 - `setRiskParameterConfig()` : Sets or updates the risk parameters for a given update type. The parameters include the Risk Steward contract address, a debounce period (a time window after the last execution time for a market update during which update registration is restricted), and a timelock period of up to 2 days, after which the update is executable.
 - `setConfigActive()` : Sets the active status of a risk parameter configuration for a given update type. When an update type is inactive, registration and execution are restricted.
 - `setWhitelistedExecutor()` : Manages a whitelist of executors.
- **Risk Steward Receiver Whitelisted Executors:** Whitelisted accounts that can execute registered updates, reject updates, and resend remote updates to a destination chain.
- **LayerZero:** Relays cross-chain messages from the `RiskStewardReceiver` on the source chain to a `DestinationStewardReceiver` on remote chains.

Destination Steward Receiver

The entry-point contract on all supported remote chains, where updates are registered, executed, or rejected.

- **Destination Steward Receiver Contract Owner:** Can update the Access Control Manager contract address; set peers for corresponding LayerZero endpoint IDs; update the delegate address, who can set LayerZero configurations directly in the endpoint contract; transfer the owner role to another account; and renounce their role, leaving the contract without an owner.
- **Destination Steward Receiver Access Control Manager Roles:**
 - `setRiskParameterConfig()` : Sets or updates the risk parameters for a given update type. The parameters include the Risk Steward contract address and a debounce period (the time window after the last execution time for a market update during which update registration is restricted).
 - `setConfigActive()` : Sets the active status of a risk parameter configuration for a given update type. When an update type is inactive, execution is restricted.
 - `setWhitelistedExecutor()` : Manages a whitelist of executors.
- **Destination Steward Receiver Whitelisted Executors:** Whitelisted accounts that can execute or register updates.
- **LayerZero:** Relays cross-chain messages from the `RiskStewardReceiver` on the source chain to a `DestinationStewardReceiver` on remote chains.

Collateral Factors Risk Steward

The risk steward contract that directly interacts with the Core Pool or Isolated Pool Comptroller to update the collateral factor and liquidation threshold for a market and applicable pool.

- **Collateral Factor Risk Steward Contract Owner:** Can update the Access Control Manager contract address and transfer the owner role to another account.
- **Collateral Factors Risk Steward Access Control Manager Roles:**
 - `setSafeDeltaBps()` : Updates the safe delta basis points up to the maximum value (100%).

Interest Rate Model Risk Steward

The risk steward contract that directly interacts with the Core Pool or Isolated Pool market to update its Interest Rate Model contract address.

- **Interest Rate Model Risk Steward Contract Owner:** Can update the Access Control Manager contract address and transfer the owner role to another account.
- **Interest Rate Model Risk Steward Access Control Manager Roles:**
 - `setSafeDeltaBps()` : Updates the safe delta basis points up to the maximum value (100%).

Market Caps Risk Steward

The risk steward contract that directly interacts with the Core Pool or Isolated Pool Comptroller to update the supply or borrow cap for a market.

- **Market Caps Risk Steward Contract Owner:** Can update the Access Control Manager contract address and transfer the owner role to another account.
- **Market Caps Risk Steward Access Control Manager Roles:**
 - `setSafeDeltaBps()` : Updates the safe delta basis points up to the maximum value (100%).

Findings

VEN-1 Multiple Debounce for Cross-Chain Calls

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `d2ac3936d4f91e082ef11b54fca6642df1099af5` .

File(s) affected: `contracts/RiskSteward/RiskStewardReceiver.sol` ,
`contracts/RiskSteward/DestinationStewardReceiver.sol`

Description: Both the source and the destination chain enforce a debounce period, during which a transaction will revert if an update to the same parameter has happened too recently.

However, the current implementation faces a few challenges:

1. The `RiskParamConfig` may enforce a different timer on each chain.
2. The `lastExecutedAt` for sending the message on the source chain and the `lastProcessedUpdate` on the destination chain may differ.
3. The debounces are restricting distinct operations. On the source chain, the debounce restricts how often a parameter update can be sent to the destination chain. On the destination chain, the debounce restricts how often a parameter update can be executed.

Having two separate debounces for the same cross-chain could lead to operational difficulty. Consolidating the debounce enforcement could help simplify the behavior and make it more predictable.

Recommendation: Consolidate the debounce enforcement for cross-chain parameter updates to only happen in the `DestinationStewardReceiver` .

VEN-2

Parameter Updates May Silently Fail Due to Unchecked Return Values

• Low ⓘ Fixed

✓ Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

This was already fixed before the audit started. <https://github.com/VenusProtocol/governance-contracts/pull/163/commits/274034f2804d4cda9830cbefca31db63657829e0>

Given the provided commit hash, the issue has been marked as Fixed.

File(s) affected: `contracts/RiskSteward/CollateralFactorsRiskSteward.sol` , `contracts/RiskSteward/IRMRiskSteward.sol`

Description: In the function `CollateralFactorsRiskSteward._updateCollateralFactors()`, the status code returned from `setCollateralFactor()` in the Core Pool Comptroller is not checked. If the call returns an error status code, the collateral factor and liquidation threshold update will fail silently. Further, the `CollateralFactorsUpdated` event is emitted even when the update fails.

Similarly, in the function `IRMRiskSteward._updateIRM()`, the status code returned from `_setInterestRateModel()` in the Core Pool Market contract is not checked.

Recommendation: Check the status code returned by `setCollateralFactor()`, and throw an error if the call is not successful.

Auditor Suggestions

S1 Inconfigurable `REMOTE_DELAY` Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `aedaeb0bad7209ab0c83342f804e3c59043f76cf`.

File(s) affected: `contracts/RiskSteward/DestinationStewardReceiver.sol`

Description: The Governance assigns the `RiskParamConfig.timelock` for which a whitelisted executor must wait before executing the registered update. In the `DestinationStewardReceiver`, the `REMOTE_DELAY` has a similar timelock functionality, but is hardcoded to 6 hours. This is intended to be stricter than any of the planned timelock durations.

However, should a timelock longer than 6 hours be desired for future updates, it the `REMOTE_DELAY` would be more lenient than the timelock on the source chain.

Recommendation: Consider making `REMOTE_DELAY` a configurable value.

S2 Missing Input Validation Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `9967888d8d09c518448ba3143f1f32e7366888c2`.

All of the suggested validation was added, with the exception of the suggestion for `RiskStewardReceiver.setRiskParameterConfig()`.

File(s) affected: `contracts/RiskSteward/DestinationStewardReceiver.sol`,
`contracts/RiskSteward/RiskStewardReceiver.sol`

Related Issue(s): [SWC-123](#)

Description: The following functions could benefit from additional input validation:

- `RiskStewardReceiver.constructor()`:
 - Validate that the `endpoint_` is a non-zero address.
 - Validate that `LAYER_ZERO_EID != 0`.
- `RiskStewardReceiver.setRiskParameterConfig()`: Validate that the `updateType` exists in `RiskOracle.allUpdateTypes()`.
- `DestinationStewardReceiver.constructor()`: Validate that `layerZeroEid_` is non-zero.

Recommendation: We recommend adding the relevant checks.

S3 Missing `renounceOwnership()` Override Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `631a04f49ab2ee7a8399680679d5ba051eab0859`.

File(s) affected: `contracts/RiskSteward/DestinationStewardReceiver.sol`

Description: The function `renounceOwnership()` (as similarly declared in `RiskStewardReceiver.sol`) is not overridden in this contract.

Recommendation: Add the `renounceOwnership()` function.

S4 RiskOracle Mappings Can Be Improved

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `1865facd2ff2a2953dcfef07ff22913bf4c9e756`.

File(s) affected: `contracts/RiskSteward/RiskOracle.sol`

Description: The `RiskOracle` has the following 2d mapping:

```
mapping(address => mapping(string => uint256)) public latestUpdateIdByMarketAndType;
```

The `RiskStewardReceiver` and `DestinationStewardReceiver` have a similar mapping:

```
mapping(bytes32 => mapping(address market => uint256)) public lastRegisteredUpdate;
```

We highlight the following improvements:

1. The keys for `RiskOracle` are reversed in position compared to the other mappings (`[market][updateType]` rather than `[updateTypeKey][market]`).
2. The `RiskOracle` has keys of type `string` rather than `bytes32`.

Recommendation:

1. Reverse the key positions for `RiskOracle.latestUpdateIdByMarketAndType` to improve code consistency.
2. Update `RiskOracle` to use `bytes32` in place of `string` as a key for its mappings.

S5 Gas issues/for-Loop Concerns

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

Both the markets in the `Comptroller` and the `updateTypes` in the `RiskOracle` will remain limited.

File(s) affected: `contracts/RiskSteward/RiskStewardReceiver.sol`, `contracts/RiskSteward/RiskOracle.sol`

Description: The function `RiskStewardReceiver.getExecutableUpdates()` iterates over every market associated with the `Comptroller`. If the number of markets can be arbitrarily large, this call could fail due to gas issues.

The function `RiskOracle._updateTypeExists()` iterates over `allUpdateTypes`, which can grow arbitrarily large.

Recommendation: Ensure that the number of markets are not too high in order to avoid gas-related issues. If a large number is expected, the function could take a "limit" parameter to short-circuit the loop.

Similarly, ensure that the number of update types in `RiskOracle` is not too large.

S6 Code Clones

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `fe4ea957b606529659379e24593daf464c21b9ef`.

The client provided the following explanation:

Receiver contracts share only few functions, and I don't expect any other contract to benefit from introducing a common base contract. It's not a strong preference, but I'd still lean toward keeping them separate for now, as this keeps the implementations clearer and easier to maintain.

Some of the code was consolidated for the Steward contracts, and the receiver contracts moved structs, events, and errors to the interfaces

File(s) affected: `contracts/RiskSteward/RiskStewardReceiver.sol` , `contracts/RiskSteward/DestinationStewardReceiver.sol` , `contracts/RiskSteward/CollateralFactorsRiskSteward.sol` , `contracts/RiskSteward/IRMRiskSteward.sol` , `contracts/RiskSteward/MarketCapsRiskSteward.sol`

Description: There are several cloned functions within the steward receiver contracts, which could be inherited from a parent class:

1. `setConfigActive()`
2. `setWhitelistedExecutor()`
3. `rejectUpdate()`
4. `getRiskParameterConfig()`
5. `transferOwnership()`
6. `_transferOwnership()`

In the three steward contracts, there are many shared events and errors, and some functions such as `_isWithinSafeDelta()` and `renounceOwnership()`

Recommendation: Consider creating parent `StewardReceiver` and `Steward` contracts that encapsulates shared functionality.

S7 Code Improvements

Mitigated

Update

Marked as "Fixed" by the client.

Addressed in: `1865facd2ff2a2953dcfef07ff22913bf4c9e756` .

The client provided the following explanation:

we are skipping point 1 as we might add or remove updateTypes in the future based on the risk stewards we support. We do not have this completely defined to make it an ENUM. Also the number of types are in our control and won't be of a large number to cause any issues.

We note that the item #4 was not implemented and `previousConfig` is still declared as `storage` .

File(s) affected: `contracts/RiskSteward/RiskOracle.sol` , `contracts/RiskSteward/DestinationStewardReceiver.sol`

Description:

1. Consider using an enumerable set for `RiskOracle.allUpdateTypes` to avoid iterating over the array in `_updateTypeExists()` .
2. In `RiskOracle._updateTypeExists()` , cache the hash of `updateType` before the `for` loop.

```
function _updateTypeExists(
    string memory updateType
) internal view returns (bool) {
    bytes32 updateTypeKey = keccak256(bytes(updateType));
    for(uint256 i = 0; allUpdateTypes.length; ++i) {
        if (keccak256(bytes(allUpdateTypes[i])) == updateTypeKey) {
            return true;
        }
    }
    return false;
}
```

3. Cache `++updateCounter` in `RiskOracle._publishUpdate()` to avoid multiple storage loads.
4. The `DestinationStewardReceiver.setRiskParameterConfig()` function declares `RiskParamConfig` storage `previousConfig` , whereas it only appears to be needed in memory.

Recommendation: Consider implementing the code improvements.

S8 Inconsistent Error Handling

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `a2612b2be6370038fcff8b9d87417b24bf354789` .

File(s) affected: `contracts/RiskSteward/RiskStewardReceiver.sol` , `contracts/RiskSteward/DestinationStewardReceiver.sol`

Description: Many of the setter functions in the `RiskStewardReceiver` and `DestinationStewardReceiver` contracts throw an error when the updated state matches the previous state. However, in following setter functions, an error is not thrown, instead the function returns:

- 1. `setConfigActive()` : Does not throw an error when `previousActive == active` .
- 2. `setWhitelistedExecutor()` : Does not throw an error when `previousApproved == approved` .
- 3. `setRiskParameterConfig()` function declares the error `UnsupportedUpdateType` in `RiskStewardReceiver` , while `DestinationStewardReceiver` uses `InvalidUpdateType` .

Recommendation: For consistency, consider throwing errors instead of returning.

S9 Misaligned Comments

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `95b8efc1dcbc10b188a369f55834020e2abe1978` .

File(s) affected: `contracts/RiskSteward/RiskStewardReceiver.sol` ,
`contracts/RiskSteward/DestinationStewardReceiver.sol` , `contracts/RiskSteward/MarketCapsRiskSteward.sol`

Description:

- 1. The comment above `LAYER_ZERO_EID` should say "Destination chain", not "Source chain".
- 2. In `MarketCapRiskSteward.sol` , there are some access control comments of the form "@custom:access Controlled by `AccessControlManager`", which are not used in other files.

Recommendation: Ensure coding styles are aligned in all contracts.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/VenusProtocol/governance-contracts`

- `4f6...214` `./contracts/RiskSteward/CollateralFactorsRiskSteward.sol`
- `727...ee9` `./contracts/RiskSteward/DestinationStewardReceiver.sol`
- `02c...32a` `./contracts/RiskSteward/IRMRiskSteward.sol`
- `b65...3ea` `./contracts/RiskSteward/Interfaces/IRiskOracle.sol`
- `6dd...e7d` `./contracts/RiskSteward/Interfaces/IRiskSteward.sol`
- `0f6...f90` `./contracts/RiskSteward/Interfaces/IRiskStewardReceiver.sol`
- `c0d...d06` `./contracts/RiskSteward/MarketCapsRiskSteward.sol`
- `883...5ae` `./contracts/RiskSteward/RiskOracle.sol`

- 2c3...f9c ./contracts/RiskSteward/RiskStewardReceiver.sol

Test Suite Results

The test suite was run with the command `npx hardhat test`.

Access Control

Access Control

- ✓ only default admin role can give call permissions (53ms)
- ✓ should **not** have permissions
- ✓ should have permissions
- ✓ should revoke role
- ✓ should be able to call the function only for the given contract
- ✓ should be able to call the function on every contract

Omnichain:

- ✓ Reverts **if** EOA called owner function of **bridge**
- ✓ Reverts **if** EOA call execute() without grant permission
- ✓ Reverts when zero value passed
- ✓ Revert **if** trusted remote is removed by non owner
- ✓ Revert **if** non trusted remote is removed
- ✓ Reverts when trusted remote is **not** set
- ✓ Reverts **if** remote address is more than 20 bytes
- ✓ Reverts with Daily Transaction Limit Exceed
- ✓ Reverts **if** EOA call setMaxDailyLimit() without grant permission
- ✓ Revert **if** function in **not** found in function registry
- ✓ Reverts **if** any user other than owner try to **add** function in function registry
- ✓ Function registry should **not** emit event **if** nonexistant function is removed
- ✓ Function registry should **not** emit event **if** function is added twice
- ✓ Reverts **if** invalid parameters passed in trusted remote
- ✓ Reverts **if** EOA called owner function of Executor
- ✓ Revert **if** call by non guardian
- ✓ Revert **if** zero address is passed in guardian
- ✓ should set **new** guardian for Executor
- ✓ Emit TimelocksAdded event (41ms)
- ✓ Emit SetTrustedRemoteAddress event (38ms)
- ✓ Emit ExecuteRemoteProposal event (38ms)
- ✓ Revert initially, success on retry (64ms)
- ✓ Revert when daily limit exceeds in retry (46ms)
- ✓ Emit ProposalExecuted event (59ms)
- ✓ Should update delay of timelock on destination (54ms)
- ✓ Admin can set the **new** pending admin of Timelock (119ms)
- ✓ Set **new** pending admin of Timelock through proposal (113ms)
- ✓ should revert when invalid proposalType is passed (71ms)
- ✓ Revert when zero address passed as pending admin
- ✓ Revert when non owner sets the pending admin of Timelock (73ms)
- ✓ Revert **if** empty proposal
- ✓ Revert on invalid proposal type
- ✓ Revert **if** same proposal come twice (57ms)
- ✓ Retry message on destination on failure (244ms)
- ✓ Revert retry message on destination **if** trusted remote is changed (104ms)
- ✓ Retry messages that failed due to low gas at the destination using the Endpoint. (50ms)
- ✓ Reverts when other than guardian call cancel of executor
- ✓ Revert **if** proposal is **not** queued
- ✓ Revert when proposal is **not** queued
- ✓ Emit ProposalCanceled event when proposal gets canceled (49ms)
- ✓ Reverts when cancel is called after **execute** (40ms)
- ✓ Proposal fails **if** any number of commands fail on destination
- ✓ Reverts when number of parameters mismatch
- ✓ Refund stucked gas in contract, to given address (47ms)
- ✓ Reverts on passing zero values in parameters in fallback withdraw
- ✓ Reverts when value exceeds contract's balance in fallback withdraw
- ✓ Reverts when different parameters passed in fallback withdraw
- ✓ Reverts when receiver is unable to receive in fallback withdraw
- ✓ Refund stucked gas in contract, to given address (52ms)
- ✓ Reverts on passing zero values in parameters in fallback withdraw (41ms)
- ✓ Reverts when different parameters passed in fallback withdraw (44ms)

- ✓ Reverts when receiver is unable to receive in fallback withdraw (42ms)
- ✓ Reverts when daily limit of sending transaction reached
- ✓ Proposal failed when receiving limit reached (77ms)

Governor Bravo Cast Vote Test

We must revert if:

- ✓ We cannot propose without enough voting power by depositing xvs to the vault after we deposit xvs to the vault

There does **not** exist a proposal with matching proposal id where the current block number is between the proposal's start block (exclusive) **and** end block (inclusive)

- ✓ Such proposal already has an entry in its voters set matching the sender

Otherwise

- ✓ we **add** the sender to the proposal's voters set

and we take the balance returned by GetPriorVotes for the given sender **and** the proposal's start block, which may be zero,

- ✓ **and** we **add** that ForVotes (56ms)

- ✓ **or** AgainstVotes corresponding to the caller's support flag. (54ms)

castVoteBySig

- ✓ reverts **if** the signatory is invalid

- ✓ casts vote on behalf of the signatory (59ms)

Governor Bravo Configuration Setter Test

Setter

- ✓ should revert **if** not called by admin
- ✓ should revert **if** setValidationParams called with invalid validationParams
- ✓ should revert **if** setProposalConfigs called with invalid proposalConfigs (70ms)
- ✓ should set correct validationParams on calling setValidationParams
- ✓ should set correct proposalConfigs on calling setProposalConfigs

Governor Bravo Initializing Test

initilizer

- ✓ should revert **if** not called by admin
- ✓ should revert **if** invalid xvs vault address
- ✓ should revert **if** invalid guardian address
- ✓ should revert **if** timelock adress count differs from governance routes count
- ✓ should revert **if** proposal config count differs from governance routes count
- ✓ should revert **if** initialized twice
- ✓ should revert **if** initialized with invalid validationParams
- ✓ should revert **if** initialized with invalid proposalConfigs
- ✓ should set correct validationParams **and** proposalConfig on initialization (47ms)

Governor Bravo Propose Tests

simple initialization

- ✓ ID is set to a globally unique identifier
- ✓ Proposer is set to the sender
- ✓ Start block is set to the current block number plus vote delay
- ✓ End block is set to the current block number plus the sum of vote delay **and** vote period
- ✓ ForVotes **and** AgainstVotes are initialized to zero
- ✓ Executed **and** Canceled flags are initialized to false
- ✓ ETA is initialized to zero
- ✓ Targets, Values, Signatures, Calldatas are set according to parameters
- ✓ This function returns the id of the newly created proposal. `# proposalId(n) = succ(proposalId(n-1))`

- ✓ emits log with id **and** description (54ms)

This function must revert **if**

- ✓ the length of the values, signatures **or** calldatas arrays are **not** the same length, (49ms)
- ✓ **or** if that length is zero **or** greater than Max Operations.

Additionally, **if** there exists a pending **or** active proposal from the same proposer, we must revert.

- ✓ reverts with pending
- ✓ reverts with active

Governor Bravo Queue Tests

overlapping actions

- ✓ reverts on queueing overlapping actions in same proposal (57ms)
- ✓ reverts on queueing overlapping actions in different proposals (116ms)

Governor Bravo State Tests

- ✓ Invalid for proposal **not** found
- ✓ Pending
- ✓ Active
- ✓ Canceled (47ms)

- ✓ Canceled by Guardian (49ms)
- ✓ Defeated
- ✓ Succeeded (58ms)
- ✓ Expired (78ms)
- ✓ Queued (92ms)
- ✓ Executed (125ms)

TimelockV8 Tests

- ✓ Production timelock returns constant values
- ✓ Production timelock requires setting appropriate delay (38ms)
- ✓ Production timelock does **not** allow a null address
- ✓ Test Timelock returns correct for constants
- ✓ Test Timelock allows setting low delay
- ✓ Test timelock does **not** allow a null address

RiskOracle

Initialization

- ✓ should initialize with access control manager (182ms)
- ✓ should start with zero update counter
- ✓ should revert when initializing with zero address (68ms)

Authorized Sender Management

- ✓ should **add** authorized sender
- ✓ should revert when adding zero address as authorized sender
- ✓ should revert when adding already authorized sender
- ✓ should revert when unauthorized user tries to **add** authorized sender
- ✓ should remove authorized sender
- ✓ should revert when removing non-authorized sender
- ✓ should revert when unauthorized user tries to remove authorized sender

Update Type Management

- ✓ should **add** update type
- ✓ should revert when adding empty update type
- ✓ should revert when adding update type exceeding 64 characters
- ✓ should revert when adding duplicate update type
- ✓ should revert when unauthorized user tries to **add** update type
- ✓ should set update type active status to false
- ✓ should set update type active status to true
- ✓ should revert when setting status for non-existent update type
- ✓ should revert when setting status to same value
- ✓ should revert when unauthorized user tries to set update type active status

Publishing Updates

- ✓ should publish single update (56ms)
- ✓ should publish update with poolId **and** dstEid
- ✓ should track previous value for subsequent updates (44ms)
- ✓ should revert when non-authorized sender tries to publish
- ✓ should revert when publishing with inactive update type
- ✓ should revert when publishing with zero address market

Bulk Publishing Updates

- ✓ should publish bulk updates (57ms)
- ✓ should revert when **array** lengths don't match
- ✓ should revert when empty **array** is provided
- ✓ should revert when any update type is inactive

Querying Updates

- ✓ should get update by ID
- ✓ should revert when getting update with invalid ID (zero)
- ✓ should revert when getting update with ID greater than counter
- ✓ should get latest update by parameter **and** market
- ✓ should revert when getting latest update for non-existent market **and** type
- ✓ should **return** all update
- ✓ should **return** empty **array** when no update types are added
- ✓ should correctly **return** all update types after deactivating one

Risk Steward

Risk Parameter Updates E2E

immediate execution

- ✓ should immediately **execute** SupplyCap update with increased value (45ms)
- ✓ should immediately **execute** BorrowCap update with decreased value (45ms)
- ✓ should immediately **execute** CollateralFactors update with increased CF **and** LT values (59ms)
- ✓ should immediately **execute** CollateralFactors update for isolated market (49ms)

pause control

- ✓ should pause, block processUpdate, **and** resume when unpaused (82ms)
- ✓ should revert for unauthorized pause attempts

registered **and** execute updates

```
      ✓ should register and then execute BorrowCap update when change exceeds safe delta and decreases
value (66ms)
      ✓ should register and then execute eMode CollateralFactor update with increased CF only (not LT)
(62ms)
      ✓ should register and then execute IRM update (68ms)
      ✓ should register and then execute IRM update for isolated market (70ms)
remote updates
      ✓ should send remote BorrowCap update to destination chain (105ms)
      ✓ should send remote CollateralFactors update for isolated market to destination chain (104ms)
      ✓ should send remote IRM update for core pool market to destination chain (114ms)
failure cases
      ✓ should revert when processing an update with inactive config
      ✓ should revert when processing an already resolved update (42ms)
      ✓ should revert when processing an expired update
      ✓ should revert when processing an update that is not the latest for the market and type
      ✓ should revert when processing an update too frequently (debounce not passed) (53ms)
      ✓ should revert when processing an update when there is a pending registered update of the same
type (49ms)
      ✓ should revert when processing remote update with unsupported update type on destination (76ms)
failure cases for executing registered updates
      ✓ should revert when trying to execute update before timelock expires (44ms)
      ✓ should revert when trying to execute an already executed update (60ms)
      ✓ should revert when trying to execute an expired registered update (41ms)
      ✓ should revert when trying to execute update with inactive config (47ms)
      ✓ should revert when non-executor tries to execute registered update
      ✓ should revert when trying to execute update that was never registered
      ✓ should revert when trying to execute a rejected update (50ms)
reject update
      ✓ should successfully reject a registered update in RSR (49ms)
      ✓ should revert when non-executor tries to reject update in RSR (39ms)
      ✓ should revert when trying to reject an already executed update in RSR (67ms)
failure cases at destination receiver
      ✓ should revert when non-executor tries to execute update on destination (71ms)
      ✓ should revert when trying to execute update with unconfigured update type on destination (74ms)
      ✓ should revert when trying to execute update that doesn't exist on destination
      ✓ should revert when trying to execute update before remote delay expires (70ms)
      ✓ should revert when trying to execute expired update on destination (69ms)
      ✓ should revert when trying to execute update too frequently on destination (debounce) (155ms)
      ✓ should revert when trying to execute already executed update on destination (87ms)
reject update at destination receiver
      ✓ should successfully reject a registered update on destination (91ms)
      ✓ should revert when non-executor tries to reject update on destination (65ms)
      ✓ should revert when trying to reject an already executed update on destination (80ms)
      ✓ should prevent execution after rejecting an update on destination (73ms)
      ✓ should revert when executing update with inactive config on destination (71ms)
getExecutableUpdates
  RiskStewardReceiver
    ✓ should return executable updates excluding rejected ones (369ms)
  DestinationStewardReceiver
    ✓ should return executable updates excluding rejected ones (444ms)

196 passing (24s)
```

Code Coverage

The test coverage was gathered with the command `npx hardhat coverage --testfiles "tests/RiskSteward/*.ts"` . We encourage the Venus team to increase the code coverage as much as possible, particularly adding unit tests for the `RiskStewardReceiver.resendRemoteUpdate()` function.

96.88	31/32	57.69	15/26	90	9/10	85.71	36/4
86.05	74/86	75	36/48	81.25	13/16	87.27	96/11
85	17/20	50	7/14	75	6/8	67.86	19/2

96.88	31/32	57.69	15/26	90	9/10	85.71	36/4
97.06	33/34	65	13/20	90	9/10	85.11	40/4
100	43/43	76.09	35/46	100	15/15	100	62/6
82.71	110/133	70.65	65/92	74.19	23/31	81.11	146/18

Changelog

- 2025-12-15 - Initial report
- 2025-12-19 - Final Report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation

provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

